

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/351744650>

Minimum-Delay Adaptation in Non-Stationary Reinforcement Learning via Online High-Confidence Change-Point Detection

Preprint · May 2021

CITATIONS

0

READS

35

3 authors:



[Lucas N. Alegre](#)

Universidade Federal do Rio Grande do Sul

15 PUBLICATIONS 67 CITATIONS

SEE PROFILE



[Ana L. C. Bazzan](#)

Universidade Federal do Rio Grande do Sul

359 PUBLICATIONS 5,963 CITATIONS

SEE PROFILE



[Bruno C. da Silva](#)

University of Massachusetts Amherst

59 PUBLICATIONS 931 CITATIONS

SEE PROFILE

Minimum-Delay Adaptation in Non-Stationary Reinforcement Learning via Online High-Confidence Change-Point Detection

Lucas N. Alegre
Institute of Informatics - Federal
University of Rio Grande do Sul
Porto Alegre, RS - Brazil
lnalegre@inf.ufrgs.br

Ana L. C. Bazzan
Institute of Informatics - Federal
University of Rio Grande do Sul
Porto Alegre, RS - Brazil
bazzan@inf.ufrgs.br

Bruno C. da Silva
CICS - University of Massachusetts
Amherst, MA
bsilva@cs.umass.edu

ABSTRACT

Non-stationary environments are challenging for reinforcement learning algorithms. If the state transition and/or reward functions change based on latent factors, the agent is effectively tasked with optimizing a behavior that maximizes performance over a possibly infinite random sequence of Markov Decision Processes (MDPs), each of which drawn from some unknown distribution. We call each such MDP a *context*. Most related works make strong assumptions such as knowledge about the distribution over contexts, the existence of pre-training phases, or *a priori* knowledge about the number, sequence, or boundaries between contexts. We introduce an algorithm that efficiently learns policies in non-stationary environments. It analyzes a possibly infinite stream of data and computes, in real-time, high-confidence change-point detection statistics that reflect whether novel, specialized policies need to be created and deployed to tackle novel contexts, or whether previously-optimized ones might be reused. We show that (i) this algorithm minimizes the delay until unforeseen changes to a context are detected, thereby allowing for rapid responses; and (ii) it bounds the rate of false alarm, which is important in order to minimize regret. Our method constructs a mixture model composed of a (possibly infinite) ensemble of probabilistic dynamics predictors that model the different modes of the distribution over underlying latent MDPs. We evaluate our algorithm on high-dimensional continuous reinforcement learning problems and show that it outperforms state-of-the-art (model-free and model-based) RL algorithms, as well as state-of-the-art meta-learning methods specially designed to deal with non-stationarity.

KEYWORDS

Reinforcement Learning; Non-stationarity; Model-based RL; Change-Point Detection

ACM Reference Format:

Lucas N. Alegre, Ana L. C. Bazzan, and Bruno C. da Silva. 2021. Minimum-Delay Adaptation in Non-Stationary Reinforcement Learning via Online High-Confidence Change-Point Detection. In *Proc. of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2021)*, Online, May 3–7, 2021, IFAAMAS, 9 pages.

1 INTRODUCTION

Reinforcement learning (RL) techniques have been successfully applied to solve high-dimensional sequential decision problems. However, if the state transition and/or reward functions change

unexpectedly, according to latent factors unobservable to the agent, the system is effectively tasked with optimizing behavior policies that maximize performance over a (possibly infinite) random sequence of Markov Decision Processes (MDPs). Each MDP is drawn from an unknown distribution and is henceforth referred to as a *context*. This is known as a non-stationary setting. Designing efficient algorithms to tackle this problem is a known challenge in RL [20]. The key difficulties here result from (i) the need to quickly and reliably detect when the underlying system dynamics has changed; and (ii) the need to effectively learn and deploy adaptable prediction models and policies, specialized in particular contexts, while allowing the agent to (when appropriate) reuse previously-acquired knowledge. Non-stationary environments often result from systems whose dynamics are inherently time-dependent; from agents that are tasked with learning policies under noisy or missing sensors; or from problems where the agent faces sequences of unlabeled/unidentified tasks, each one with its own transition dynamics and reward functions.

Non-stationary settings arise naturally in many situations. Humans, for instance, are capable of learning to solve sequences of tasks from few experiences while preserving knowledge from older experiences [13]. Consider, for example, a person realizing the need to adapt their gait after an accident, learning novel gait patterns to use when walking with crutches, and then, after a period of recovery, successfully re-deploying normal walking gaits. This corresponds to a non-stationary scenario where the agent needs to learn specialized dynamics models and policies for tackling different contexts/learning scenarios.

We introduce an algorithm that efficiently learns decision-making strategies in this setting. It assumes that an agent experiences random sequences of contexts (MDPs) drawn from some unknown distribution, and it is capable of optimizing behaviors even when a pre-training phase (during which the agent interacts with sample contexts) is not available. The agent’s goal is to rapidly deploy policies that are appropriate for each randomly-arriving context—even when the number of latent contexts is unknown and when the context distribution cannot be modeled nor controlled by the agent. We are particularly interested in the case where quick readaptation and sample efficiency are paramount to achieving good performance; for instance, in novel contexts where collecting experiences and acquiring policies from scratch is unfeasible.

Many existing related works tackle non-stationary problems either by detecting when the underlying MDP changes, or via meta-learning approaches that construct a prior model (or policy) capable of rapidly generalizing to novel contexts. Hadoux et al., for example, introduced a technique based on change-point detection algorithms

to deal with non-stationary problems with discrete state spaces [2, 9]. We, by contrast, address the more general setting of high-dimensional continuous RL problems. Supervised meta-learning algorithms [6] have also been recently combined with RL to enable fast adaptation under changing domains [18, 19]. Nagabandi et al. introduced a model-based algorithm where a meta-learning technique is used to construct dynamics models capable of rapidly adapting to recent experiences—either by updating the hidden state of a recurrent neural network [5], or by updating the model parameters via a small number of gradient steps [6]. Meta-learning methods typically assume disjoint training and testing phases, so that an agent can be pre-trained over randomly sampled contexts prior to its deployment. We, however, do not require a pre-training phase. Meta-learning methods also typically assume that the distribution over contexts experienced during training is the same as the one experienced during testing, so that agents can adapt to novel environments with structural similarities to those previously experienced. We, by contrast, do not require that contexts are sampled from a previously-seen distribution, nor that contexts share structural similarities with previously-experienced ones.

To address these limitations, we introduce an algorithm that analyzes a possibly infinite stream of data and computes, in real-time, high-confidence change-point detection statistics that reflect whether novel, specialized policies need to be deployed to tackle new contexts, or whether a previously-optimized policy may be reused. We call our algorithm Model-Based RL Context Detection, or MB_{CD}. We formally show that it minimizes the delay until unforeseen changes to a context are detected, thereby allowing for rapid responses, and that it allows for formal bounds on the rate of false alarm—which is of interest when minimizing the agent’s regret over random sequences of contexts. Our method constructs a mixture model composed of a (possibly infinite) ensemble of probabilistic dynamics predictors that model the different modes of the distribution over underlying latent MDPs. Our method is capable of optimizing policies based on streams of arbitrarily different contexts, with unknown boundaries, and which may be drawn from an arbitrary, unknown distribution.

We evaluate our algorithm on high-dimensional continuous reinforcement learning tasks and empirically show (i) that state-of-the-art reinforcement learning algorithms struggle to deal with non-stationarity; and (ii) that our method outperforms state-of-the-art meta-learning methods specifically tailored to deal with non-stationary environments—in particular, when the agent is faced with MDPs that are off-distribution with respect to the set of training contexts provided to the agent, or when novel contexts are structurally different from previously-observed ones.

2 PROBLEM FORMULATION

We define a non-stationary environment as a family of MDPs $\{\mathcal{M}_z\}_{z \in \mathbb{N}^+}$. Each MDP $\mathcal{M}_z$ is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}_z, \mathcal{R}_z, \gamma, d^0)$, where $\mathcal{S}$ is a (possibly continuous) state space, $\mathcal{A}$ is a (possibly continuous) action space, $\mathcal{T}_z : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is a transition function specifying the distribution over next states, given the current state and action, $\mathcal{R}_z : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a reward function, $\gamma \in [0, 1]$ is the discount factor, and d^0 is an initial state distribution. In what follows, S_t , A_t , and R_t are the random variables corresponding to

the state, action, and reward at time step t . We assume that the agent observes a random sequence $(\mathcal{M}_0, \mathcal{M}_1, \dots)$ of MDPs—called *contexts*—drawn independently from some unknown distribution. We assume that the number of contexts, $|\{\mathcal{M}_z\}|$, is unknown. These definitions are similar to those discussed in [2] and [20]. Let z be a latent index variable indicating a particular MDP, $\mathcal{M}_z$. We assume that each MDP’s transition and reward function are parameterized by a latent vector θ_z . Let $p_{\theta_z}(S_{t+1}, R_t | S_t, A_t)$ denote the joint conditional probability distribution over next-state and reward associated with MDP $\mathcal{M}_z$. We do not impose any smoothness assumptions on how variations to θ_z affect $\mathcal{T}_z$ and $\mathcal{R}_z$: contexts may be arbitrarily different and share no structural similarities.

Let the time steps in which context changes occur be an increasing sequence of integer random variables, $\{C_i\}_{i \geq 1}$, for which a prior $\phi(C_i)$ is unknown or cannot be defined. We call each C_i a *change-point*. At every change-point C_i , the current context $\mathcal{M}_z$ is replaced by a new randomly drawn MDP. To perform well, an agent must rapidly detect context changes and deploy an appropriate policy. If a new random context differs significantly from previously-experienced ones, the agent may have to learn a policy from scratch; otherwise, it may choose to reuse previously-acquired knowledge to accelerate learning and avoid catastrophic forgetting.

At each time t , when interacting with its environment, the agent selects an action A_t based on its state S_t according to a stochastic policy $\pi(\cdot | S_t)$. Let $v_{\pi, \mathcal{M}_z}^H$ be the value function associated with policy π , MDP $\mathcal{M}_z$, and defined over a horizon H :

$$v_{\pi, \mathcal{M}_z}^H(s) = \mathbb{E} \left[\sum_{j=0}^{H-1} \gamma^j R_{t+j} | S_t = s, \pi, \mathcal{M}_z \right]. \quad (1)$$

To simplify our analysis, we first consider (without loss of generality) the simpler case of a family of MPDs $\{\mathcal{M}_0, \mathcal{M}_1\}$, where a context change occurs from $\mathcal{M}_0$ to $\mathcal{M}_1$ at some unobserved random time C . The mathematical arguments that follow can be extended to the more general setting with an arbitrary number of contexts. Let Π^* be a policy that follows the optimal policy for $\mathcal{M}_0$, π_0 , before C , and the optimal policy for $\mathcal{M}_1$, π_1 , afterwards. This policy’s value function is defined as:

$$v_{\Pi^*, \{\mathcal{M}_0, \mathcal{M}_1\}}^\infty(s) = \mathbb{E} \left[v_{\pi_0, \mathcal{M}_0}^C(s) + \gamma^C \mathbb{E}_{s' \sim \rho_{\mathcal{M}_0, C-1}^{\pi_0}} \left[v_{\pi_1, \mathcal{M}_1}^\infty(s') \right] \right], \quad (2)$$

where $\rho_{\mathcal{M}_0, t}^\pi$ denotes the distribution of states reachable after following policy π for t steps under MDP $\mathcal{M}$. Notice that Eq. 2 models the case when $\mathcal{M}_1$ starts in the (random) state where $\mathcal{M}_0$ terminated, that is, immediately prior to the random time C . This implies that $d_{\mathcal{M}_1}^0 = \rho_{\mathcal{M}_0, C-1}^{\pi_0}$.

By contrast, consider an alternative policy Π that follows policy π_1 only after a random time Γ , for $\Gamma > C$; that is, a policy that deploys the correct decision-making strategy for $\mathcal{M}_1$ with a delay of $\Delta = \Gamma - C$ steps. Its value function is given by:

$$v_{\Pi, \{\mathcal{M}_0, \mathcal{M}_1\}}^\infty(s) = \mathbb{E} \left[v_{\pi_0, \mathcal{M}_0}^C(s) + \gamma^C \mathbb{E}_{s' \sim \rho_{\mathcal{M}_0, C-1}^{\pi_0}} \left[v_{\pi_0, \mathcal{M}_1}^\Delta(s') \right] + \gamma^{C+\Delta+1} \mathbb{E}_{s' \sim \rho_{\mathcal{M}_1, \Delta}^{\pi_0}} \left[v_{\pi_1, \mathcal{M}_1}^\infty(s') \right] \right]. \quad (3)$$

Notice that we can rewrite Eq. 2 in the same form as Eq. 3:

$$v_{\Pi^*, \{\mathcal{M}_0, \mathcal{M}_1\}}^\infty(s) = \mathbb{E}[v_{\pi_0, \mathcal{M}_0}^C(s) + \gamma^C \mathbb{E}_{s' \sim \rho_{\mathcal{M}_0, C-1}^{\pi_0}} \left[v_{\pi_1, \mathcal{M}_1}^\Delta(s') \right] + \gamma^{C+\Delta+1} \mathbb{E}_{s' \sim \rho_{\mathcal{M}_1, \Delta}^{\pi_1}} \left[v_{\pi_1, \mathcal{M}_1}^\infty(s') \right]]. \quad (4)$$

We now define the regret $\mathcal{L}(\Delta)$ as the expected discounted sum of rewards lost due to the delay $\Delta = \Gamma - C$, when changing from π_0 to π_1 only at time step Γ . This quantity is given by the difference between Eq. 3 and Eq. 4. Since we are interested in minimizing the delay Δ , we assume the adversarial case when policies π_0 and π_1 do, in the short-term (i.e., within the delay window Δ), have a nearly indistinguishable distribution over the states that are reachable in Δ steps. In particular, we assume that the KL divergence between $\rho_{\mathcal{M}_1, \Delta}^{\pi_0}$ and $\rho_{\mathcal{M}_1, \Delta}^{\pi_1}$ is bounded and small for small values of Δ . In this case, the regret can be approximated by¹:

$$\mathcal{L}(\Delta) \approx \mathbb{E} \left[\gamma^C \mathbb{E}_{s' \sim \rho_{\mathcal{M}_0, \Gamma-1}^{\pi_0}} \left[v_{\pi_1, \mathcal{M}_1}^\Delta(s') - v_{\pi_0, \mathcal{M}_1}^\Delta(s') \right] \right] \quad (5)$$

as $\mathcal{D}_{KL}(\rho_{\mathcal{M}_1, \Delta}^{\pi_0} \| \rho_{\mathcal{M}_1, \Delta}^{\pi_1}) \rightarrow 0$. From Eq. 5, it should be clear that to maximize the expected return over the random sequence of MDPs, one needs to minimize regret; and to minimize regret, one needs to minimize the random delay Δ . In the next section, we introduce a method capable of minimizing the worst-case delay until unforeseen changes to a stochastic process are detected, while also bounding the rate of false alarm—i.e., the likelihood that the method will *incorrectly* indicate that a context change occurred.

3 HIGH-CONFIDENCE CHANGE-POINT DETECTION

Change-point detection (CPD) algorithms [1, 30] are designed to detect whether (and when) a change occurs in the distribution generating random observations from an arbitrary stochastic process. These methods have been widely used in a variety of fields—from financial markets [15] to biomedical signal processing [26]. Although CPDs have been applied to reinforcement learning problems [2, 9], the application and formal analysis of such methods have been restricted to discrete state spaces settings.

In the *online* CPD setting, a sequential detection procedure is defined to rapidly and reliably estimate when the parameter θ of some underlying distribution or stochastic process has changed. Online CPD algorithms should produce high-confidence estimates, Γ , of the true change-point time, C . Notice that Γ is a random variable whose stochasticity results from the unknown stochastic prior over context changes, ϕ , and from the fact that each MDP in $\{\mathcal{M}_z\}$ produces random trajectories of states, actions, and rewards.

Suppose that at each time t , while the agent interacts with $\mathcal{M}_0$, sample next-state and rewards are drawn from $p_{\theta_0}(S_{t+1}, R_t | S_t, A_t)$, where θ_0 is the latent vector parameterizing $\mathcal{M}_0$'s transition and reward functions. At some unknown random change-point C , the context changes to $\mathcal{M}_1$, and experiences that follow are drawn

from $p_{\theta_1}(S_{t+1}, R_t | S_t, A_t)$. We propose to identify such a change by computing high-confidence statistics that reflect whether θ_0 has changed. This can be achieved by introducing a minimax formulation of the CPD problem, as discussed by Pollak [22]. In this formulation, the goal of a CPD algorithm is to compute a random estimate, Γ , of the latent change-point time C , such that (i) it minimizes the worst-case expected detection delay, $\Delta_{\text{worst}}(\Gamma)$, associated with the random estimates Γ produced by a particular CPD algorithm, when considering all possible change-points C ; and (ii) bounds on the maximum false alarm rate (FAR) may be imposed. The worst-case expected detection delay, $\Delta_{\text{worst}}(\Gamma)$, and the FAR, are defined as:

$$\Delta_{\text{worst}}(\Gamma) = \sup_{c \geq 1} \mathbb{E}[\Gamma - C | \Gamma \geq C, C = c], \quad (6)$$

$$\text{FAR}(\Gamma) = \frac{1}{\mathbb{E}[\Gamma | C = \infty]}, \quad (7)$$

where the expectations in Eq. 6 and Eq. 7 are over the possible histories of experiences produced by the stochastic process, and where conditioning on $C = \infty$ indicates the random event where the context never changes. Given these definitions, the objective of a high-confidence change-point detection process is the following:

$$\inf_{\Gamma} \Delta_{\text{worst}}(\Gamma) \text{ subject to } \text{FAR}(\Gamma) \leq \alpha, \quad (8)$$

where α denotes the desired upper-bound on the false alarm rate.

When θ_0 and θ_1 are known, the Log-Likelihood Ratio (LLR) statistic can be used to recursively compute the CUSUM statistic [21]. As we will discuss next, such a statistic can be used to construct a high-confidence change-point detection method. The LLR statistic, L_t , and the CUSUM statistic, W_t , are updated at each time t as follows:

$$L_t = \log \frac{p_{\theta_1}(S_{t+1}, R_t | S_t, A_t)}{p_{\theta_0}(S_{t+1}, R_t | S_t, A_t)}, \quad (9)$$

$$W_t = \max(0, W_{t-1} + L_t), \quad W_0 = 0. \quad (10)$$

Importantly, notice that before the change-point C is reached, $\mathbb{E}[L_t] < 0$, which implies that the expected value of W_t is zero. After the change-point C is reached, $\mathbb{E}[L_t] > 0$, and therefore W_t will tend to increase. Higher values of W_t , thus, serve as principled statistics reflecting evidence that a change-point has occurred between θ_0 to θ_1 . The random time Γ when a change-point is estimated to have happened is defined as the first time when the CUSUM metric W_t becomes greater than a detection threshold h :

$$\Gamma = \min\{t \geq 1 : W_t > h\}. \quad (11)$$

In [16], Lorden shows that choosing $h = |\log \alpha|$ ensures that $\text{FAR}(\Gamma) \leq \alpha$. Furthermore, Lai demonstrated that the CUSUM detection time Γ is asymptotically optimum [12] with respect to the problem specified in Eq. 6. In particular, they showed that the worst expected detection delay (under $h = |\log \alpha|$) respects the following approximation:

$$\Delta_{\text{worst}}(\Gamma) \approx \frac{|\log \alpha|}{\mathcal{D}_{KL}(p_{\theta_1} \| p_{\theta_0})} \text{ as } \alpha \rightarrow 0. \quad (12)$$

In Eq. 12, the denominator $\mathcal{D}_{KL}(p_{\theta_1} \| p_{\theta_0}) = \mathbb{E}_{\theta_1} \left[\log \frac{p_{\theta_1}(S_{t+1}, R_t | S_t, A_t)}{p_{\theta_0}(S_{t+1}, R_t | S_t, A_t)} \right]$ is the Kullback–Leibler divergence under θ_1 . Eq. 12 implies that the larger the difference between the distributions p_{θ_1} and p_{θ_0} , the smaller the expected delay ($\Delta = \Gamma - C$) for detecting a change-point. The above results allow us to construct high-confidence statistics

¹This definition can be extended in a straightforward way to the case where there is a sequence of random change-points, $\{C_i\}_{i \geq 1}$. In particular, the regret will be defined in terms not of a single random delay, but of a sequence of random delays associated with the corresponding random contexts that are observed by the agent.

reflecting whether (and when) a context has changed; importantly, they are both accurate and have bounded false alarm rate.

4 MODEL-BASED RL CONTEXT DETECTION

In this section, we introduce an algorithm that iteratively applies a CUSUM-related procedure to detect context changes under the assumptions discussed in Section 2. The algorithm incrementally builds a library of models and policies for tackling arbitrarily different types of contexts; i.e., contexts that may result from quantitatively and qualitatively different underlying causes for non-stationarity—ranging from unpredictable environmental changes (such as random wind) to robot malfunctions. Our method can rapidly deploy previously-constructed policies whenever contexts approximately re-occur, or learn new decision-making strategies whenever novel contexts, with no structural similarities with respect to previously-observed ones, are first encountered. Unlike existing approaches (see Section 6), our method is capable of (i) optimizing policies *online*; (ii) without requiring a pre-training phase; (iii) based on streams of arbitrarily different contexts, with unknown change-point boundaries; and (iv) such that contexts may be drawn from an arbitrary, unknown distribution.

We now introduce a *high-level* description of our method (Model-Based RL Context Detection, or MBCD). In subsequent subsections, we provide details for each of the method’s components. As the agent interacts with a non-stationary environment, context changes are identified via a multivariate variant of CUSUM [10], called MCUSUM. MCUSUM-based statistics inherit the same formal properties as those presented in Section 3. In particular, they formally guarantee that MBCD can detect context changes with minimum expected delay, while simultaneously bounding the false alarm rate. As a consequence, MBCD can effectively identify novel environmental dynamics while ensuring, with high probability, that new context-specific policies will only be constructed when necessary.

As new contexts are identified by this procedure, MBCD updates a mixture model, M , composed of a (possibly infinite) ensemble of probabilistic context dynamics predictors, whose purpose is to model the different modes of the distribution over underlying latent MDPs/contexts. New models are added to the ensemble as qualitatively different contexts are first encountered. The mixture model M associates, with each identified context $\mathcal{M}_z$, a learned joint distribution p_{θ_z} over next-state dynamics and rewards associated. Let K be the number of context models currently in the mixture. After each agent experience, MBCD identifies the most likely context, z_t , by analyzing a set of incrementally-estimated MCUSUM statistics (see Section 4.2). Whenever a novel context—one with dynamics that are qualitatively different from those previously-experienced—is observed, a new model is added to the mixture. Context-specific policies, π_{ψ_z} , are trained via a Dyna-style approach [28] based on the corresponding learned joint prediction model of $\mathcal{M}_z$, p_{θ_z} (see Section 4.3). We provide details in the next subsections. Pseudocode for MBCD is shown in Algorithm 1.

4.1 Stochastic Mixture Model of Dynamics

In this paper, we assume that p_{θ_z} , the joint distribution over next-state dynamics and rewards associated with context $\mathcal{M}_z$, can be approximated by a multivariate Gaussian distribution. In particular,

following recent work on model-based RL [4, 11], MBCD models the dynamics of a given environment $\mathcal{M}_z$, $p_{\theta_z}(S_{t+1}, R_t | S_t, A_t)$, via a bootstrap ensemble of probabilistic neural networks whose outputs parameterize a multivariate Gaussian distribution with diagonal covariance matrix. The bootstrapping procedure accounts for epistemic uncertainty (i.e. uncertainty about model parameters), which is crucial when making predictions about the agent’s dynamics in regions of the state space where experiences are scarce. For each context $\mathcal{M}_z$ identified by MBCD, an ensemble of N stochastic neural networks is instantiated and added to the mixture model M . Each network n in the ensemble is parameterized by θ_z^n and computes a probability distribution, $p_{\theta_z^n}$, that approximates p_{θ_z} by predicting the mean and covariance over next-state and rewards conditioned on the current state and action:

$$p_{\theta_z^n}(S_{t+1}, R_t | S_t, A_t) = \mathcal{N}(\mu_{\theta_z^n}(S_t, A_t), \Sigma_{\theta_z^n}(S_t, A_t)), \quad (13)$$

where $\mu_{\theta_z^n}(S_t, A_t)$ and $\Sigma_{\theta_z^n}(S_t, A_t)$ are the network outputs given input (S_t, A_t) . To simplify notation, let $\mathbf{X}_t = (S_t, A_t) \in \mathbb{R}^{\dim(S) + \dim(\mathcal{A})}$ and $\mathbf{Y}_t = (S_{t+1}, R_t) \in \mathbb{R}^{\dim(S)+1}$. We follow Lakshminarayanan et al. [14] and model the ensemble prediction as a Gaussian distribution whose mean and covariance are computed based on the mean and covariances of each component of the ensemble. In particular, the ensemble predictive model, $\hat{p}_{\theta_z}$, associated with a given context $\mathcal{M}_z$, is defined as:

$$\hat{p}_{\theta_z}(\mathbf{Y}_t | \mathbf{X}_t) = \mathcal{N}(\mu_z^*(\mathbf{X}_t), \Sigma_z^*(\mathbf{X}_t)), \quad (14)$$

where

$$\begin{aligned} \mu_z^*(\mathbf{X}_t) &= N^{-1} \sum_{n=1}^N (\mu_{\theta_z^n}(\mathbf{X}_t)), \text{ and} \\ \Sigma_z^*(\mathbf{X}_t) &= N^{-1} \sum_{n=1}^N (\text{diag}(\Sigma_{\theta_z^n}(\mathbf{X}_t)) + \mu_{\theta_z^n}^2(\mathbf{X}_t)) - \mu_z^{*2}(\mathbf{X}_t). \end{aligned} \quad (15)$$

MBCD stores all experiences collected while in context $\mathcal{M}_z$ in a buffer D_z . After every F steps, it uses data in D_z to update the ensemble model $\hat{p}_{\theta_z}$ by minimizing the negative log prediction likelihood loss function, $\mathcal{J}_p(\theta, D) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim D} [-\log p_{\theta}(s_{t+1}, r_t | s_t, a_t)]$.

4.2 Online Context Change-Point Detection

As previously discussed, MBCD employs a multivariate variant of CUSUM, MCUSUM, to detect context changes with high confidence. Healy demonstrated that, when detecting shifts in the mean of a multivariate Gaussian distribution, MCUSUM inherits all theoretical optimality guarantees possessed by the univariate CUSUM procedure [10]. Furthermore, he also proved that the detection delay is independent of the dimensionality of the data.

In the particular case where the dynamics of each context are modeled as multivariate Gaussians, the LLR statistic can be computed as follows. To simplify notation, let $\mu_0 = \mu_{\theta_0}(S_t, A_t)$ and $\Sigma_0 = \Sigma_{\theta_0}(S_t, A_t)$. It is then possible to show that the LLR statistic, L_t , between distributions $p_{\theta_1}(\mathbf{Y}_t | \mathbf{X}_t)$ and $p_{\theta_0}(\mathbf{Y}_t | \mathbf{X}_t)$, is given by:

$$L_t = \log \frac{(2\pi)^{-\frac{d}{2}} |\Sigma_1|^{-\frac{1}{2}} \exp\{-0.5(\mathbf{Y}_t - \mu_1)\Sigma_1^{-1}(\mathbf{Y}_t - \mu_1)\}}{(2\pi)^{-\frac{d}{2}} |\Sigma_0|^{-\frac{1}{2}} \exp\{-0.5(\mathbf{Y}_t - \mu_0)\Sigma_0^{-1}(\mathbf{Y}_t - \mu_0)\}} \quad (17)$$

where d is the dimensionality of the multivariate Gaussian. At each time step t , MBCD uses L_t to compute MCUSUM statistics $W_{k,t}$ for

each known context k , plus an additional statistic $W_{\text{new},t}$, used to infer, with high probability, whether a novel context has been first encountered:

$$W_{k,t} \leftarrow \max \left(0, W_{k,t-1} + \log \frac{p_{\theta_k}(\mathbf{Y}_t | \mathbf{X}_t)}{p_{\theta_{\text{new}}}(\mathbf{Y}_t | \mathbf{X}_t)} \right), \quad \forall k \in [1, K] \cup [\text{new}]. \quad (18)$$

Here, $W_{\text{new},t}$ can be seen as evidence that a previously-unseen context has been first encountered, based on whether the likelihood of all known contexts $k \in [1, K]$ is smaller than $p_{\theta_{\text{new}}}$, where

$$p_{\theta_{\text{new}}}(\mathbf{Y}_t | \mathbf{X}_t) = \mathcal{N}(\hat{\mathbf{Y}}_t, \Sigma_{\theta_{z_t}}(\mathbf{X}_t)), \quad (19)$$

and

$$\hat{\mathbf{Y}}_t = \mathbf{Y}_t + \delta \text{diag}(\Sigma_{\theta_{z_t}}(\mathbf{X}_t)). \quad (20)$$

Intuitively, $W_{\text{new},t}$ indicates whether none of the known contexts is likely to have generated the observed transitions. In Eq. 19, $p_{\theta_{\text{new}}}$ is the likelihood of a new context under the alternative hypothesis that the true observation $\mathbf{Y}_t$ is δ standard deviations away from the true observation $\mathbf{Y}_t$. In particular, δ indicates the minimum meaningful change in the distribution’s mean that we are interested in detecting. Given updated statistics $W_{k,t}$, the most likely current context, z_t (which may or may not have changed) can then be identified as:

$$z_t \leftarrow \begin{cases} \text{argmax}_k W_{k,t}, & \text{if } \exists k \in [1, K] \cup [\text{new}] \text{ s.t. } W_{k,t} > h, \\ z_{t-1}, & \text{otherwise.} \end{cases} \quad (21)$$

If no alternative contexts are more likely to have generated the observations collected up to time t , no context change is detected and $z_t = z_{t-1}$.

Notice that in Eq. 18, models p_{θ_k} are assumed to be known *a priori*. In our setting, these models are estimated based on samples. MCUSUM has been studied in scenarios where the parameters of the distribution are known only approximately [17]. In our work, we address this challenge by computing change-point detection statistics only after a small warm-up period within which the agent is allowed to operate in a given context. In particular, and similarly to Sekar et al. [25], we define the warm-up period by using ensemble disagreement as a proxy to quantify the system’s uncertainty regarding the current distributions. Assuming a warm-up period where the agent is allowed to operate within each newly-encountered context is a common assumption in the area [18, 19, 23]. In fact, it is a *necessary* assumption: if contexts are allowed to change arbitrarily fast, adversarial settings can be constructed where all methods for dealing with non-stationary scenarios fail.

Finally, notice that a key element of Eq. 21 is the threshold h , against which each $W_{k,t}$ is compared in order to check if a context change has occurred. Different methods have been proposed to set h [24]. Here, we take a conservative approach. As discussed in Section 3, setting $h = |\log \alpha|$ ensures that $\text{FAR}(\Gamma) \leq \alpha$. In this work, we set h by considering negligible values of α ; e.g. $h = 100$ if $\alpha \approx 10^{-43}$. In our experiments, we observed that detection delays remain low even for very conservative values of h .

4.3 Policy Optimization

Since MBCD estimates dynamics models for each context, it is natural to exploit such models to accelerate policy learning by deploying model-based RL algorithms. MBCD learns context-specific policies

via a Dyna-style approach [28]. In particular, at every time step t during which the model $p_{\theta_{z_t}}$ is trained, L 1-step simulated rollouts are sampled using $\pi_{\psi_{z_t}}$. Each rollout starts from a random state drawn from D_{z_t} . All rollouts are stored in a buffer, D_{model} .² Notice that this is similar to the procedure used by the Model-Based Policy Optimization (MBPO) algorithm [11]. Each context-specific policy $\pi_{\psi_{z_t}}$ is optimized by taking into account both real experiences (stored in D_{z_t}) and simulated experiences (stored in D_{model}). Policy optimization is performed using the Soft Actor-Critic (SAC) algorithm [8]. SAC alternates between a policy evaluation step, which estimates $q_{\pi}(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R_t | S_t = s, A_t = a, \pi]$ using the Bellman backup operator, and a policy improvement step, which optimizes the policy π by minimizing the expected KL-divergence between the current policy and the exponential of a soft Q-function [8]. Optimizing the policy, then, corresponds to minimizing the following loss function:

$$\mathcal{J}_{\pi}(\psi, D) = \mathbb{E}_{s_t \sim D} \left[\mathbb{E}_{a_t \sim \pi_{\psi}} (\beta \log(\pi_{\psi}(a_t | s_t)) - q_{\pi_{\psi}}(s_t, a_t)) \right]. \quad (22)$$

Algorithm 1: Model-Based RL Context Detection

Input: Non-stationary environment E , threshold h .

```

1  $z_0 \leftarrow 1; K \leftarrow 1; W_{z_0,0} \leftarrow 0; W_{\text{new},0} \leftarrow 0$ 
2 Initialize model  $p_{\theta_{z_0}}$ , policy  $\pi_{\psi_{z_0}}$ , datasets  $D_{z_0}$  and  $D_{\text{model}}$ 
3  $M \leftarrow \{p_{\theta_{z_0}}\}$ 
4 for  $t = 0 \dots \infty$  do
5   Execute action  $a_t \sim \pi_{\psi_{z_t}}(s_t)$ , observe  $s_{t+1}, r_t$ 
6   Update MCUSUM statistics  $W_{k,t}, \forall k$ , with Eq. 18
7   Update  $z_t$  with Eq. 21
8   if  $z_t \neq z_{t-1}$  then // Context changed
9     Reset MCUSUM statistics
10     $D_{\text{model}} = \{\}$ 
11    if  $z_t = \text{new}$  then // New context detected
12       $K \leftarrow K + 1; z_t \leftarrow K$ 
13      Initialize  $p_{\theta_{z_t}}$  and  $D_{z_t}$ 
14      Let  $\pi_{\psi_{z_t}} \leftarrow \pi_{\psi_{z_{t-1}}}$ 
15       $M \leftarrow M \cup \{p_{\theta_{z_t}}\}$ 
16     $D_{z_t} \leftarrow D_{z_t} \cup \{(s_t, a_t, r_t, s_{t+1})\}$ 
17    if  $t \bmod F = 0$  then
18       $\theta_{z_t} \leftarrow \theta_{z_t} - \lambda_p \nabla \mathcal{J}_p(\theta_{z_t}, D_{z_t})$  // Update model
19      for  $L$  simulated 1-step rollouts do
20        Sample state  $s_i$  from tuples in  $D_{z_t}$ 
21         $a_i \sim \pi_{\psi_{z_t}}(\cdot | s_i)$ 
22         $(s'_i, r_i) \sim p_{\theta_{z_t}}(\cdot | s_i, a_i)$ 
23         $D_{\text{model}} \leftarrow D_{\text{model}} \cup (s_i, a_i, r_i, s'_i)$ 
24       $\psi_{z_t} \leftarrow \psi_{z_t} - \lambda_{\pi} \nabla \mathcal{J}_{\pi}(\psi_{z_t}, D_{\text{model}} \cup D_{z_t})$  // Update policy
```

5 EXPERIMENTS

We evaluate MBCD in challenging continuous-state, continuous-action non-stationary environments, where the non-stationarity

²Notice that, in Algorithm 1, D_{model} is cleared every time a context change is detected in order to avoid negative transfer from experiences drawn from previous contexts.

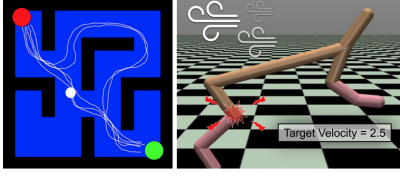


Figure 1: (a) Non-Stationary Continuous Particle Maze; (b) Half-Cheetah in a Non-Stationary World.

may result from qualitatively different reasons—ranging from abrupt changes to the system’s dynamics (such as changes to the configuration of the agent’s workspace); external latent environmental factors that impact the distribution over next states (such as random wind); robot malfunctions; and changes to the agent’s goals (its reward function). We compare MBCD both against state-of-the-art RL algorithms and against state-of-the-art meta-learning methods specifically tailored to deal with non-stationary environments.

We investigate the performance of MBCD in two settings: (i) a setting that satisfies all standard requirements made by meta-learning algorithms; in particular, that all contexts in M_z are structurally similar, and that the agent is tested on a distribution of contexts that matches the training distribution; and (ii) a more general setting where such requirements are not be satisfied: contexts may differ arbitrarily, and future contexts experienced by the agent may be off-distribution with respect to those sampled during the training phase. We show that our method outperforms both standard state-of-the-art RL algorithms and also specialized meta-learning algorithms in both settings.

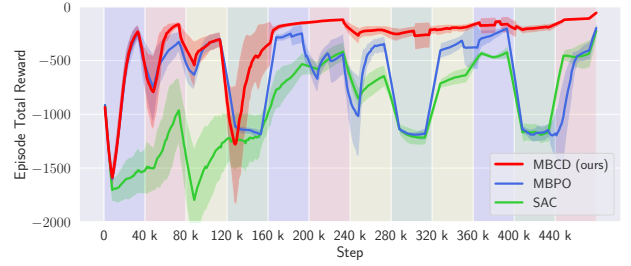
In what follows, we evaluate MBCD in two domains with qualitatively different non-stationary characteristics (see Fig. 1):

Non-Stationary Continuous Particle Maze. This domain simulates a family of two-dimensional continuous mazes where a particle must reach a non-observable goal location. The reward function corresponds to the Euclidean distance between the particle and the goal location. Non-stationarity is introduced by either changing the location of walls or by randomly changing the latent goal location.

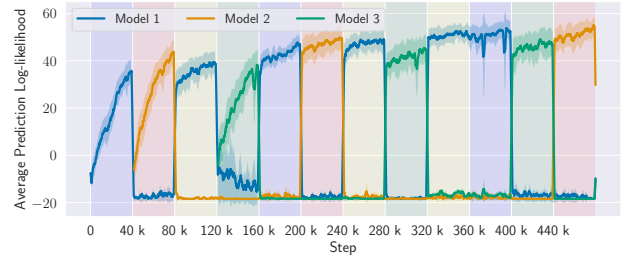
Half-Cheetah in a Non-Stationary World. This domain consists of a simulation of the high-dimensional *Half-Cheetah* robot [29]. The agent’s goal is to move forward while reaching a particular target-velocity and minimizing control costs. We introduce three sources of non-stationarity:

- (1) **random wind**: an external latent horizontal force, opposite to the agent’s movement direction, is applied;
- (2) **joint malfunction**: either the torque applied to a joint of the robot has its polarity/sign changed; or a joint is randomly disabled;
- (3) **target velocity**: the target velocity of the robot is sampled from the interval 1.5 to 2.5, causing a non-stationary change to the agent’s reward function.

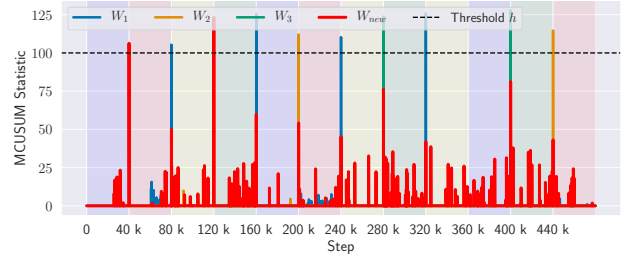
We first evaluate our method on the non-stationary Half-Cheetah domain and compare it with two state-of-the-art RL algorithms: MBPO [11] and SAC [8]. In our setting, MBPO can be seen as a particular case of our algorithm, where a single dynamics model and policy are tasked with optimizing behavior under changing



(a) Total reward achieved by different methods (MBCD, MBPO, and SAC) as contexts change.



(b) Log-likelihood of the predictions made by each context model learned by MBCD as contexts change.



(c) Time evolution, as contexts change, of the MCUSUM statistics, W_k , for each model k . A context change is detected online whenever one of the statistics crosses the threshold h .

Figure 2: Evaluation of MBCD on the non-stationary *Half-Cheetah* domain. Colored shaded areas represent different contexts: (blue) default context; (red) joint malfunction; (yellow) wind; (green) novel target velocity.

contexts. SAC works similarly to MBPO but does not perform Dyna-style planning steps using a learned dynamics model.

Fig. 2a shows the total reward achieved by different methods (ours, MBPO, SAC) as contexts change. Colored shaded areas depict different contexts, as discussed in the figure’s caption. Notice that our method and MBPO have similar performances when interacting for the first time with the first three random contexts. In particular, both MBCD and MBPO’s performances temporarily drop when a novel context is encountered for the first time. MBCD’s performance drops because it instantiates a new dynamics model for the newly encountered context, while MBPO’s performance drops because it undergoes negative transfer. SAC, which

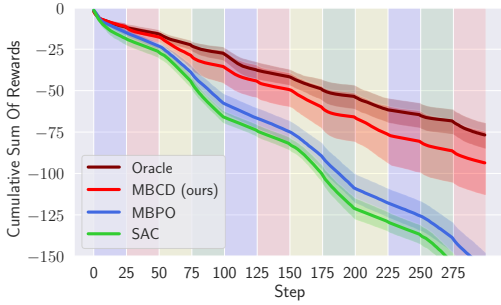


Figure 3: Regret of different methods (MBCD, MBPO, SAC) in the non-stationary *Half-Cheetah* domain, as contexts change rapidly, compared to an Oracle algorithm.

is model-free, never manages to achieve reasonable performance during the duration of each context, due to sample inefficiency. However, as the agent encounters contexts with structural similarities with respect to previously-encountered ones (around time step 160k), MBCD’s performance becomes near-optimal: it rapidly identifies whenever a context change has occurred and deploys an appropriate policy.³ MBPO and SAC, on the other hand, suffer from negative transfer due to learning average policies or dynamics models. They are also subject to catastrophic forgetting and do not reuse previously-acquired, context-specific knowledge.

Fig. 2b and Fig. 2c allow us to observe the inner workings of MBCD and understand the reasons that underlie its performance. Fig. 2b shows the log-likelihood of the predictions made by each context model learned by MBCD as contexts change. Notice that all context changes in this experiment—even those caused by qualitatively different sources of non-stationarity—are detected with minimal delay. As contexts change, MBCD rapidly detects each change and instantiates new specialized joint prediction models for each context. Furthermore, when structurally similar contexts are re-encountered (e.g., at time steps 160k and timestep 360k), MBCD successfully recognizes that previously-learned models may be re-deployed and avoids having to relearn context-specific dynamics or policies. Fig. 2c shows the time evolution, as contexts change, of the MCUSUM statistics, W_k , for each model k in the ensemble. A context change is detected, online, whenever one of the statistics crosses the threshold h . Notice that MCUSUM statistics grow rapidly and cross the threshold almost instantaneously—only a few steps after a context change. This empirically confirms the minimum-delay guarantees provided by our change-point detection method.

We now evaluate MBCD’s ability to rapidly detect context changes after an initial training period. Fig. 3 shows the cumulative sum of rewards achieved by MBCD, MBPO, SAC, and by an Oracle algorithm that is initialized with optimal policies for all contexts and that detects context changes with zero delay. This is a challenging setting where contexts change very rapidly—after only 25

³Notice that MBCD modeled the wind context (yellow area) using the same model as the default context (blue area). This is because wind did not introduce a significant change to the MDPs state transition function. Consequently, MBCD automatically inferred that a single policy could perform well in both contexts and operated without significant reward loss in the long term—see, e.g., time steps 320k to 400k.

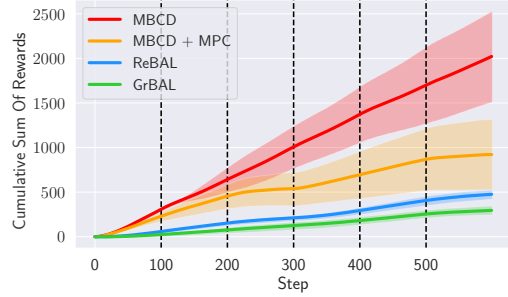


Figure 4: Performance of MBCD and meta-learning methods (after a pre-training phase) in the *Half-Cheetah* domain with non-stationary malfunctions that disable random joints. Vertical dashed lines indicate context changes.

steps. Notice that our algorithm closely matches the performance of the zero-delay Oracle, thus empirically confirming its ability to minimize regret (Eq. 5).

Next, we analyze how MBCD performs when compared with state-of-the-art meta-learning methods specifically tailored to deal with non-stationary environments: Gradient-Based Adaptive Learner⁴ (GrBAL) [18] and Recurrence-Based Adaptive Learner⁴ (ReBAL) [18]. GrBAL employs the Model-Agnostic Meta-Learning (MAML) [6] method to learn the parameters of a meta-learning prior over the dynamics model, given a set of training contexts. This prior is constructed so that it serves as a good initial model for any new contexts that the agent encounters after a pre-training phase. After training, such a meta-learned dynamics model is capable of quickly adapting to a current task’s dynamics by taking only a few gradient steps. ReBAL works similarly to GrBAL, but instead of taking gradient steps to adapt a prior model to novel contexts, it uses a recurrent neural network that learns its own update rule (vs. a gradient update rule) through its hidden state. Both ReBAL and GrBAL use Model-Predictive Control (MPC) [7] for selecting actions by planning for a certain horizon using the learned dynamics model.

Fig. 4 compares the adaptation performance of MBCD and the meta-learning methods in a non-stationary setting where (inspired by [18]) random joints of the *Half-Cheetah* robot are disabled after every 100 time steps. In this experiment we compare MBCD, ReBAL, GrBAL, and also (for fairness) a variant of MBCD that chooses actions using MPC instead of SAC. All implementations of MPC make use of the cross entropy method (CEM) [3] to accelerate action selection. All methods are allowed to interact with each randomly-sampled context during a training phase comprising 60000 time steps. Although the meta-learning methods have lower-variance, their meta-prior models do not perform as well as the MBCD context-specific dynamics models and policies. We also observe that when MBCD uses parameterized policies, learned through Dyna-style planning, it performs better than MBCD coupled with MPC.

We now evaluate MBCD in a setting where contexts may differ arbitrarily and where future contexts may be off-distribution

⁴We used the authors’ implementation of the method, publicly available at https://github.com/iclavera/learning_to_adapt.

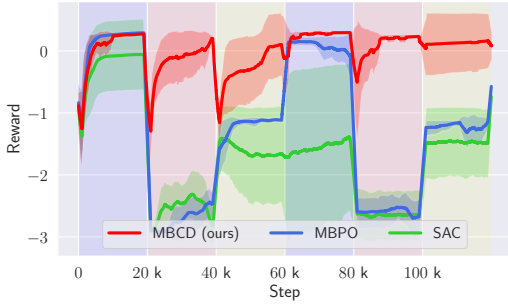


Figure 5: Rewards in the non-stationary continuous particle maze domain. Shaded colored areas indicate different contexts: (blue) default maze; (red) maze with non-stationary wall positions; (yellow) non-stationary target positions.

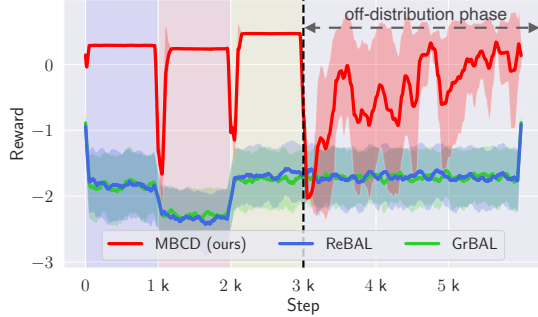


Figure 6: Rewards in the non-stationary maze domain. We introduce a phase with off-distribution contexts—contexts unlike those observed during pre-training.

with respect to those sampled during the training phase. To do this, we compare MBCD, MBPO, SAC, ReBAL, and GrBAL, in the non-stationary continuous particle maze domain, where the sources of non-stationarity are as discussed earlier. Fig. 5 compares MBCD, MBPO, and SAC in the fully-online setting—no pre-training phase is allowed. Notice that, even in this relatively simple scenario, state-of-the-art RL algorithms may fail if the underlying state transition or reward function changes drastically. MBCD’s performance, by contrast, remains approximately constant (and high) as contexts change. In Fig. 6, we compare MBCD with meta-learning methods after a phase of pre-training. All contexts observed up to time step 3k are on-distribution: they are similar to those experienced during training. MBCD outperforms ReBAL and GrBAL because the meta-learning approaches construct models that try to average characteristics of structurally different contexts. At time 3k, we initiate a phase with off-distribution contexts—contexts unlike those observed during training. When this occurs, MBCD faces a short adaption period and degrades gracefully, while meta-learning techniques perform poorly. This emphasizes MBCD’s advantages over meta-learning models, both when a pre-training phase is not allowed/possible (e.g., Fig 5) and also when testing contexts arise from a distribution different from the training distribution.

6 RELATED WORK

Dealing with non-stationary environments in RL via context change-point detection has been studied in discrete state and action spaces settings. da Silva et al. [27] introduced Reinforcement Learning Context Detection (RLCD). RLCD is a model-based algorithm that estimates the prediction quality of different models, and instantiates new ones when none of the existing models performs well. Even though it does not require a pre-training phase (like meta-learning algorithms), it is only applicable to purely discrete settings. Hadoux et al. introduced an extension of RLCD that uses a CUSUM-based method to perform change-point detection [9]. Unlike our method, however, it is only applicable to discrete state settings. Banerjee et al. proposed a two-threshold switching policy based on KL divergence between transition models in order to rapidly detect context changes [2]. This is a principled method but—unlike MBCD—requires prior knowledge of the dynamics model of all contexts.

In [18] and [19], meta-learning algorithms are used to train a prior over dynamics models that can, when combined with recent data, be rapidly adapted to novel contexts. These methods, unlike MBCD, were designed to tackle settings where the non-stationarity solely results from changes to the dynamics, but not to the agent’s goals/reward function. Reward functions are assumed to be known *a priori*. These methods also require an explicit pre-training train phase, prior to deployment, and assume that the distribution of training and testing contexts is the same. Our method, by contrast, is better suited to continual online settings where a pre-training phase is not possible, and where the agent is tasked with dealing with streams of arbitrarily different contexts with unknown boundaries.

7 CONCLUSION

We introduced a model-based reinforcement learning algorithm (MBCD) that learns efficiently in non-stationary settings with continuous states and actions. It makes use of high-confidence change-point detection statistics to detect context changes with minimum delay, while bounding the rate of false alarm. It is capable of optimizing policies online, without requiring a pre-training phase, even when faced with streams of arbitrarily different contexts drawn from unknown distributions. We empirically show that it outperforms state-of-the-art (model-free and model-based) RL algorithms, and that it outperforms state-of-the-art meta-learning methods specially designed to deal with non-stationarity—in particular, if the agent is faced with MDPs that are off-distribution with respect to the set of training contexts. As future work, we would like to extend our method so that it can actively transfer knowledge of learned policies between contexts. This research direction suggests that our method may be combined with meta-learning techniques that operate over policies, instead of over dynamics models.

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001. Ana Bazzan was partially supported by CNPq under grant no. 307215/2017-2.

REFERENCES

- [1] Samaneh Aminikhanghahi and Diane Cook. 2016. A Survey of Methods for Time Series Change Point Detection. *Knowledge and Information Systems* 51 (Sept. 2016).
- [2] Taposh Banerjee, Miao Liu, and Jonathan P. How. 2017. Quickest change detection approach to optimal control in Markov decision processes with model changes. In *2017 American Control Conference (ACC)*. IEEE, Seattle, WA, USA, 399–405.
- [3] Zdravko I. Botev, Dirk P. Kroese, Reuven Y. Rubinstein, and Pierre L’Ecuyer. 2013. Chapter 3 - The Cross-Entropy Method for Optimization. In *Handbook of Statistics*. Handbook of Statistics, Vol. 31. Elsevier, 35–59.
- [4] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. 2018. Deep Reinforcement Learning in a Handful of Trials Using Probabilistic Dynamics Models. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS’18)*. Montréal, Canada, 4759–4770.
- [5] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. 2016. RL²: Fast Reinforcement Learning via Slow Reinforcement Learning. *arXiv e-prints* (Nov. 2016). arXiv:1611.02779 [cs.AI]
- [6] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, Sydney, Australia, 1126–1135.
- [7] Carlos E. García, David M. Prett, and Manfred Morari. 1989. Model predictive control: Theory and practice—A survey. *Automatica* 25, 3 (1989), 335–348.
- [8] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, Stockholm, Sweden, 1861–1870.
- [9] Emmanuel Hadoux, Aurélie Beynier, and Paul Weng. 2014. Sequential Decision-Making under Non-stationary Environments via Sequential Change-point Detection. In *Learning over Multiple Contexts (LMCE)*. Nancy, France.
- [10] John D. Healy. 1987. A Note on Multivariate CUSUM Procedures. *Technometrics* 29, 4 (1987), 409–412.
- [11] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. 2019. When to Trust Your Model: Model-Based Policy Optimization. In *Advances in Neural Information Processing Systems (NIPS) 32*. Curran Associates, Inc., Vancouver, Canada, 12519–12530.
- [12] Tze Leung Lai. 1998. Information bounds and quick detection of parameter changes in stochastic systems. *IEEE Transactions on Information Theory* 44, 7 (1998), 2917–2929.
- [13] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. 2015. Human-level concept learning through probabilistic program induction. *Science* 350, 6266 (2015), 1332–1338.
- [14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation Using Deep Ensembles. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS) (NIPS’17)*. California, USA, 6405–6416.
- [15] Kin Lam and H. Yam. 1997. Cusum Techniques for Technical Trading in Financial Markets. *Asia-Pacific Financial Markets* 4 (Jan. 1997), 257–274.
- [16] Gary Lorden. 1971. Procedures for Reacting to a Change in Distribution. *The Annals of Mathematical Statistics* 42 (Dec. 1971), 1897–1908.
- [17] Mahmoud A. Mahmoud and Petros E. Maravelakis. 2013. The performance of multivariate CUSUM control charts with estimated parameters. *Journal of Statistical Computation and Simulation* 83, 4 (2013), 721–738.
- [18] Anusha Nagabandi, Ignasi Clavera, Simin Liu, Ronald S. Fearing, Pieter Abbeel, Sergey Levine, and Chelsea Finn. 2019. Learning to Adapt in Dynamic, Real-World Environments through Meta-Reinforcement Learning. In *7th International Conference on Learning Representations (ICLR)*. New Orleans, LA, USA.
- [19] Anusha Nagabandi, Chelsea Finn, and Sergey Levine. 2019. Deep Online Learning via Meta-Learning: Continual Adaptation for Model-Based RL. In *7th International Conference on Learning Representations (ICLR)*. New Orleans, LA, USA.
- [20] Sindhu Padakandla. 2020. A Survey of Reinforcement Learning Algorithms for Dynamically Varying Environments. *arXiv e-prints* (May 2020). arXiv:2005.10619 [cs.LG]
- [21] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115.
- [22] Moshe Pollak. 1985. Optimal Detection of a Change in Distribution. *The Annals of Statistics* 13 (March 1985), 206–227.
- [23] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. 2019. Efficient Off-Policy Meta-Reinforcement Learning via Probabilistic Context Variables. In *Proceedings of the 36th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, Long Beach, California, USA, 5331–5340.
- [24] Nassim Sahki, Anne Gegout-Petit, and Sophie Wantz-Mézières. 2020. Performance study of change-point detection thresholds for cumulative sum statistic in a sequential context. *Quality and Reliability Engineering International* 36, 8 (July 2020), 2699–2719.
- [25] Ramanan Sekar, Oleh Rybkin, Kostas Daniilidis, Pieter Abbeel, Danijar Hafner, and Deepak Pathak. 2020. Planning to Explore via Self-Supervised World Models. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, Virtual, 8583–8592.
- [26] Thabani Sibanda and Nokuthaba Sibanda. 2007. The CUSUM chart method as a tool for continuous monitoring of clinical outcomes using routinely collected data. *BMC medical research methodology* 7 (Nov. 2007), 46.
- [27] Bruno Castro da Silva, Eduardo W. Basso, Ana L. C. Bazzan, and Paulo M. Engel. 2006. Dealing with Non-Stationary Environments using Context Detection. In *Proceedings of the 23rd International Conference on Machine Learning ICML*. New York, ACM Press, Pittsburgh, USA, 217–224.
- [28] Richard S. Sutton. 1990. Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming. In *Proceedings of the 7th International Conference on Machine Learning (ICML)*. Austin, TX, USA, 216–224.
- [29] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS’12)*. Vilamoura, Portugal, 5026–5033.
- [30] Venugopal Veeravalli and Taposh Banerjee. 2012. Quickest Change Detection. *Academic Press Library in Signal Processing* 3 (Oct. 2012), 209–255.

SUPPLEMENTAL MATERIAL

1 Environments

We evaluate MBCD in two domains with qualitatively different non-stationary characteristics (see Fig. 1).

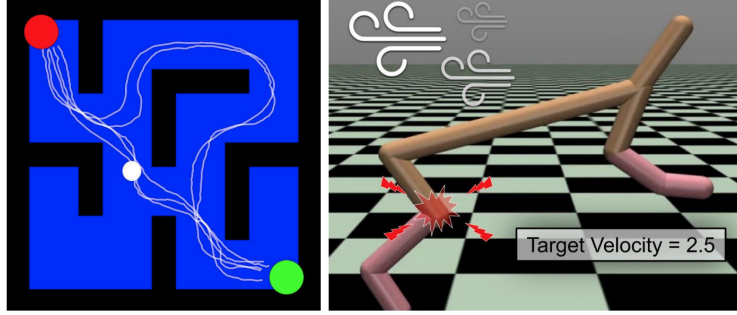


Figure 1: (a) Non-Stationary Continuous Particle Maze; (b) Half-Cheetah in a Non-Stationary World.

1.1 Non-Stationary Continuous Particle Maze

This domain simulates a family of two-dimensional continuous mazes where a particle must reach a non-observable goal location.

- **Observation.** $S_t = [x_{pos}, y_{pos}]$. The observed state consists of the particle coordinates in the continuous grid.
- **Action.** $A_t \in [-1.0, 1.0]^2$. Represents the direction of movement along the two dimensions.
- **Reward.** $R_t = -||S_t - goal||_2$. The reward function corresponds to the negative of the Euclidean distance between the particle and the goal location. The agent also receives a bonus reward of +1 when it is near the goal location.

Non-stationarity is introduced by either changing the location of walls or by randomly changing the latent goal location.

1.2 Half-Cheetah in a Non-Stationary World

This domain consists in a simulation of the high-dimensional *Half-Cheetah* robot from OpenAI Gym (Brockman et al. 2016), using MuJoCo’s physics engine (Todorov et al. 2012). *Half-Cheetah* agent is made up of 7 rigid links (1 for torso, 3 for forelimb, and 3 for hindlimb), connected by 6 joints.

- **Observation.** $S_t \in \mathbb{R}^{17}$. The observed state is given by a 17-dimensional real-valued vector, including the position and velocity of the agent’s center of mass and the angular position and angular velocity of each of its 6 joints.
- **Action.** $A_t \in [-1.0, 1.0]^6$. The action vector is the torque applied to each one of the agent’s 6 joints.
- **Reward.** $R_t = -|v_x - v_g| - 0.1||A_t||^2$, where v_x is the agent’s observed velocity along the x-axis, and v_g is the target velocity. The agent’s goal is to move forward while reaching a target-velocity and minimizing control costs.

We introduce three sources of non-stationarity:

1. **random wind:** an external latent horizontal force, opposite to the agent’s movement direction, is applied;
2. **joint malfunction:** either the torque applied to a joint of the robot has its polarity/sign changed; or a joint is randomly disabled;

3. **target velocity**: the target velocity of the robot is sampled from the interval 1.5 to 2.5, causing a non-stationary change to the agent’s reward function.

2 Parameter Settings

In Table 1 we show the parameters used for MBCD in the *Half-Cheetah* and the *Continuous Particle Maze* domains.

Table 1: MBCD parameters.

Environment	Half-Cheetah	Continuous Particle Maze
N	5	5
Dynamics architecture	4 layers with 200 neurons	2 layers with 32 neurons
F	250	250
L	10^5	10^5
Policy architecture	2 layers with 256 neurons	2 layers with 64 neurons
h	100	1000
δ	2	2.5

The neural networks used to model both the dynamics and the policies are Multi-Layer Perceptrons (MLP) with Rectified Linear Units (ReLU) activation function (Nair, 2010). They were trained with mini-batch gradient descent using the Adam optimizer (Kingma & Ba, 2015).

The parameters used for SAC and MBPO were the same (when applied) parameters as in Table 1.

For GrBAL and ReBAL, we used a reference implementation provided by the authors¹. The parameters used for the *Half-Cheetah* domain were the same as in the original paper. A reduced number of layers and neurons were used for the *Continuous Particle Maze* domain. Regarding the action selection using MPC, we used the CEM method with 1000 candidate actions and planning horizon equal to 20 for all methods.

In all figures depicting our experimental results, each curve shows the mean and shaded areas present information about the standard deviation, when running the experiments with different random seeds. We used 20 different random seeds in all experiments, except for the experiments in Fig. 2 and Fig. 5 in which we used 7 random seeds.

The code for MBCD is available at <https://github.com/LucasAlegre/mbcd>.

¹https://github.com/iclavera/learning_to_adapt